

Algorithms

Lecture #23

Syed Hamed Raza

Greedy Algorithm

Greedy Algorithms

Optimization Problems

Optimization problems

- An optimization problem is one in which you want to find, not just a solution, but the best solution.
- Search techniques look at many possible solutions. E.g. dynamic programming or backtrack search.
- A “greedy algorithm” sometimes works well for optimization problems

Optimization Problems

Optimization problems

A greedy algorithm works in phases.
At each phase:

- You take the best you can get right now, without regard for future consequences.
- You hope that by choosing a local optimum at each step, you will end up at a global optimum.

Optimization Problems

Optimization problems

- For some problems, greedy approach always gets optimum.
- For others, greedy finds good, but not always best. If so, it is called a greedy heuristic, or approximation.
- For still others, greedy approach can do very poorly.

Example: Counting Money

Example: Counting money

- Suppose you want to count out a certain amount of money, using the fewest possible bills (notes) and coins.
- A greedy algorithm would do this would be: At each step, take the largest possible bill or coin that does not overshoot

Example: Counting Money

Example: Counting money

- A greedy algorithm would do this would be: At each step, take the largest possible bill or coin that does not overshoot

```
while (N > 0){  
  give largest denomination coin  $\leq$  N  
  reduce N by value of that coin  
}
```

Coin Change

Coin Change

Formally, the Coin Change problem is:

- Given k denominations $d_1, d_2, \dots, d_k$ and given N , find a way of writing

$$N = i_1 d_1 + i_2 d_2 + \dots + i_k d_k$$

- such that $i_1 + i_2 + \dots + i_k$ is minimized.
- The “size” of problem is k .

Example: Counting Money

Example: Counting Money

Example: To make \$6.39, you can choose:

- a \$5 note
- a \$1 note to make \$6
- a 25 cents coin (quarter), to make \$6.25
- a 10 cents coin (dime), to make \$6.35
- four 1 cents coins, to make \$6.39

A failure of the Greedy Algorithm

A failure of the greedy algorithm

- In some (fictional) monetary system, "krons" come in 1 kron, 7 kron, and 10 kron coins
- Using a greedy algorithm to count out 15 krons, you would get
 - A 10 kron piece
 - Five 1 kron pieces, for a total of 15 krons
- This requires six coins

A failure of the Greedy Algorithm

A failure of the greedy algorithm

- A better solution would be to use two 7 kron pieces and one 1 kron piece
- This only requires three coins
- The greedy algorithm results in a solution, but not in an optimal solution

Greedy Algorithm for Coin Change

Greedy algorithm for coin change

- The greedy approach gives us an optimal solution when the coins are all powers of a fixed denomination.
$$N = i_0 D^0 + i_1 D^1 + i_2 D^2 + \dots + i_k D^k$$
- Note that this is N represented in based D .
- U.S.A coins are multiples of 5:
 - 5 cents – nickle
 - 10 cents – dime
 - 25 cents – quarter

Making Change DP Solution

Making Change: DP Solution

- Set up a Table, $C[1..k, 0..N]$
- Rows denote available denominations, d_i ; ($1 \leq i \leq k$)
- Columns denote the amount from $0 \dots N$ units, ($0 \leq j \leq N$).
- $C[i, j]$ denotes the minimum number of coins, required to pay an amount j using only coins of denominations 1 to i .
- $C[k, N]$ is the solution required.

Making Change DP Solution

Making Change: DP Solution

To pay an amount j units, using coins of denominations 1 to i , we have two choices:

1. either chose NOT to use any coins of denomination i ,
2. or chose at least one coin of denomination i , and also pay the amount $(j - d_i)$.

Making Change DP Solution

Making Change: DP Solution

- To pay $(j - d_i)$ units it takes $C[i, j - d_i]$ coins.
- Thus,

$$C[i, j] = 1 + C[i, j - d_i]$$

- Since we want to minimize the number of coins used,

$$C[i, j] = \min(C[i - 1, j], 1 + C[i, j - d_i])$$

DP Algorithm for Making Change

DP Algorithm for making change

COINS(N)

1 $d[1..n] = \{1, 4, 6\}$ // (coinage, for example)

2 for $i = 1$ to k

3 do $c[i, 0] \leftarrow 0$

4

DP Algorithm for Making Change

DP Algorithm for making change

COINS(N)

1 //continued

2 for $i = 1$ to k

3 do for $j = 1$ to N

4 do if ($i = 1$ & $j < d[i]$)

5 then $c[i, j] \leftarrow \infty$

6 else if ($i = 1$)

7 then $c[i, j] \leftarrow 1 + c[1, j - d[1]]$

8 else if ($j < d[i]$)

9 then $c[i, j] \leftarrow c[i - 1, j]$

DP Algorithm for Making Change

DP Algorithm for making change

```
4 do if ( $i = 1$  &  $j < d[i]$ )  
5 then  $c[i, j] \leftarrow \infty$   
6 else if ( $i = 1$ )  
7 then  $c[i, j] \leftarrow 1 + c[1, j - d[1]]$   
8 else if ( $j < d[i]$ )  
9 then  $c[i, j] \leftarrow c[i - 1, j]$   
10 else  $c[i, j] \leftarrow \min(c[i - 1, j], 1 + c[i, j - d[i]])$   
  
11 return  $c[k, N]$ 
```

Complexity of Coin Change

Complexity of Coin Change

- Greedy algorithm (non-optimal) takes $O(k)$ time.
- Dynamic Programming takes $O(kN)$ time. Note that N can be as large as 2^k so DP is really exponential in k .

Coin Change

Design and Analysis of Algorithms